



Lab #10: Interrupts of 8086

Objective

- To introduce the students to the concept of interrupts.
- Use pre-defined interrupt routines to read from and write to the console.

Introduction

In this lab, you will be introduced to the concept of interrupts. An interrupt is a hardware-initiated subroutine call. This gives the hardware access to software without any interaction until the hardware needs software to perform some function.

Interrupts

Interrupts	Extra Registers	Description	Example
INT 21h / AH=9	DX = Offset of msg data label	Output a string at DS:DX, String must be terminated by '\$'	<pre>.model small .stack 64 .data DATA_ASC DB "Hello World", "\$" newline db 13, 10, "\$" .code main proc far mov ax, @data mov ds, ax MOV AH, 09 ; option 09 to display string of data MOV DX, OFFSET DATA_ASC ;DX offset address of data INT 21H ;invoke the interrupt mov ah, 4ch int 21h end main</pre>
INT 21h / AH=2	DL = Character to write (ASCII value)	Outputting a single character to the monitor	<pre>.model small .stack 64 .data .code main proc far mov ax, @data mov ds, ax MOV AH, 02 ; option 02 displays one character</pre>

			<pre>MOV DL,"Y" ; DL holds the character to be displayed INT 21H ; invoke the interrupt. mov ah, 4ch int 21h end main</pre>																																																			
INT 21h / AH=1	Character is stored in AL	Inputting a single character, with echo	<pre>.model small .stack 64 .data .code main proc far mov ax, @data mov ds, ax MOV AH, 01 INT 21H mov ah, 4ch int 21h end main</pre>																																																			
INT 10h / AH = 09h	AL = character to display. BH = page number. BL = attribute. CX = number of times to write character.	write character and attribute at cursor position Values for attribute: <table><tr><th>HEX</th><th>BIN</th><th>COLOR</th></tr><tr><td>0</td><td>0000</td><td>black</td></tr><tr><td>1</td><td>0001</td><td>blue</td></tr><tr><td>2</td><td>0010</td><td>green</td></tr><tr><td>3</td><td>0011</td><td>cyan</td></tr><tr><td>4</td><td>0100</td><td>red</td></tr><tr><td>5</td><td>0101</td><td>magenta</td></tr><tr><td>6</td><td>0110</td><td>brown</td></tr><tr><td>7</td><td>0111</td><td>light gray</td></tr><tr><td>8</td><td>1000</td><td>dark gray</td></tr><tr><td>9</td><td>1001</td><td>light blue</td></tr><tr><td>A</td><td>1010</td><td>light green</td></tr><tr><td>B</td><td>1011</td><td>light cyan</td></tr><tr><td>C</td><td>1100</td><td>light red</td></tr><tr><td>D</td><td>1101</td><td>light magenta</td></tr><tr><td>E</td><td>1110</td><td>yellow</td></tr><tr><td>F</td><td>1111</td><td>white</td></tr></table>	HEX	BIN	COLOR	0	0000	black	1	0001	blue	2	0010	green	3	0011	cyan	4	0100	red	5	0101	magenta	6	0110	brown	7	0111	light gray	8	1000	dark gray	9	1001	light blue	A	1010	light green	B	1011	light cyan	C	1100	light red	D	1101	light magenta	E	1110	yellow	F	1111	white	<pre>.model small .stack 64 .data .code main proc far mov ax, @data mov ds, ax MOV AH, 09 MOV AL, 'S' MOV BH, 0 MOV BL, 1 ;Prints in blue MOV CX, 1 INT 10H mov ah, 4ch int 21h end main</pre>
HEX	BIN	COLOR																																																				
0	0000	black																																																				
1	0001	blue																																																				
2	0010	green																																																				
3	0011	cyan																																																				
4	0100	red																																																				
5	0101	magenta																																																				
6	0110	brown																																																				
7	0111	light gray																																																				
8	1000	dark gray																																																				
9	1001	light blue																																																				
A	1010	light green																																																				
B	1011	light cyan																																																				
C	1100	light red																																																				
D	1101	light magenta																																																				
E	1110	yellow																																																				
F	1111	white																																																				

Daoud's Lab Question:

```
.model small  
.stack 64
```

```
.data  
str1 db "My name is $"  
str2 db "mohamed$"  
NEWLINE db 13,10,"$"
```

```
.code  
main:  
mov ax, @data  
mov ds, ax
```

```
mov ah, 09  
lea dx, str1  
int 21h  
mov ah, 09  
lea dx, str2  
int 21h
```

```
mov si, offset str2  
mov cx, 7
```

```
again:  
    mov bh, [si]  
    cmp bh, "A"  
    jb skip1  
    cmp bh, "Z"  
    ja skip1  
    add bh, 32
```

```
skip1:  
    cmp bh, "a"  
    jb skip2  
    cmp bh, "z"  
    ja skip2  
    sub bh, 32
```

```
skip2:  
    mov [si], bh  
    inc si  
    loop again
```

```
mov ah, 09  
lea dx, NEWLINE  
int 21h
```

```
mov ah, 09  
lea dx, str1  
int 21h
```

```

mov ah, 09
lea dx, str2
int 21h

```

```

mov ah, 4ch
int 21h

```

```

end main

```

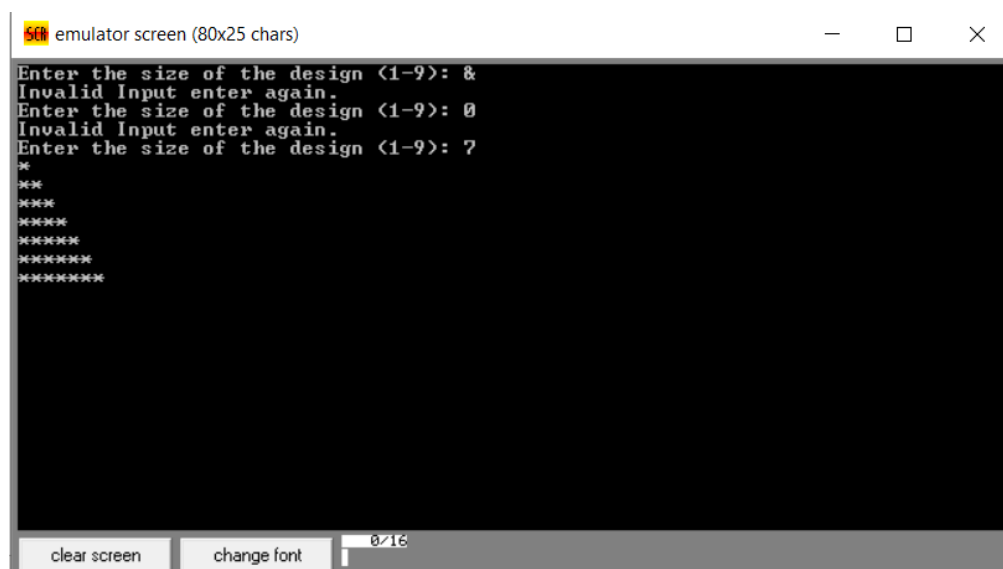
```

My name is mohamed
My name is MOHAMED

```

Lab Work (1-2):

1. Write an assembly program that first asks the user the size of the design (it is limited from 1 to 9) and then print the following half pyramid pattern. **Hint: To convert the input from ascii to actual value you must subtract the value with '0'. Use NEWLINE db 13, 10, "\$" in the datasegment for newline.**



Code:

```

.model small
.stack 64

.data
str1 db "Enter the size of the design <1-9>: $"
str2 db "Invalid Input enter again.$"
star db "$"
NEWLINE db 13,10,"$"

.code
main:
mov ax, @data
mov ds, ax

```

```
;clearing the screen
MOV AX,0600H ;scroll the entire page
MOV BH,07 ;normal attribute
MOV CX,0000 ; CL=CH=0 -> row and column of the top left
MOV DX,184FH ;CH = 24D ,DL =79D -> row and column of the bottom right
INT 10H ;invoke interrupt 10H
```

```
;setting the cursor to the center of the screen
MOV AH,02 ;set cursor option
MOV BH,00 ;page 0
MOV DH,0 ;center row position
MOV DL,0 ;center column position
INT 10H ;invoke interrupt 10H
```

```
mov cx, 10
```

repeat:

```
lea dx, str1; pointer to the message
mov ah, 09h ; print the message
int 21h
mov ah, 01 ; user input
int 21h
sub al, '0'
cmp al, 1
jl failCondi
cmp al, 9
jg failCondi
```

```
jmp success
```

failCondi:

```
lea dx, NEWLINE
mov ah, 09h
int 21h
lea dx, str2
mov ah, 09h
int 21h
lea dx, NEWLINE
mov ah, 09h
int 21h
loop repeat
```

success:

```
mov ah,0
mov cx, ax
mov bx, 1
```

outer:

```
lea dx, NEWLINE
mov ah, 09h
int 21h
```

```
push cx
mov cx, bx
```

inner:

```

lea dx, star
mov ah, 09h
int 21h
loop inner

lea dx, NEWLINE
mov ah, 09h
int 21h

pop cx
inc bx

loop outer

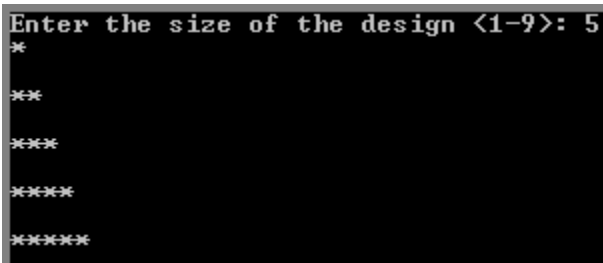
done:

mov ah, 4ch
int 21h

end main

```

Screenshot:

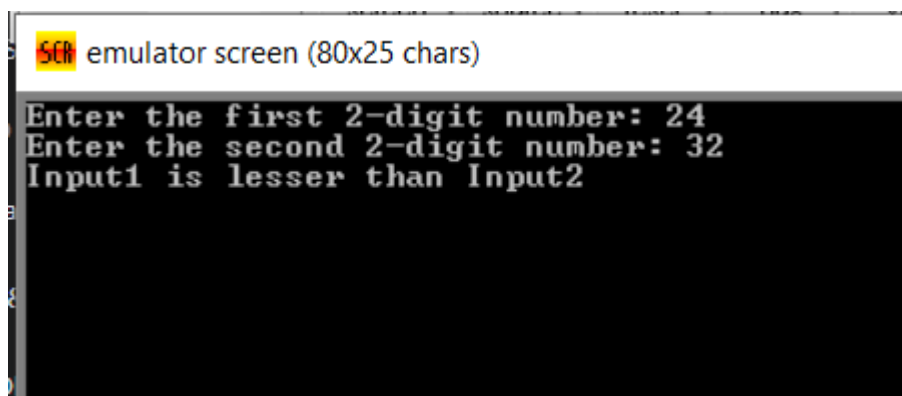


```

Enter the size of the design <1-9>: 5
*
**
***
****
*****

```

- Write a program that asks the user to enter two 2 digit numbers and prints out whether num1 is greater than num2, num1 is lesser than num2 or that the two numbers are equal. You may assume the user only enter correct values. Make sure to submit 3 samples of outputs.



```

SCM emulator screen (80x25 chars)
Enter the first 2-digit number: 24
Enter the second 2-digit number: 32
Input1 is lesser than Input2

```

Code:

```

.model small
.stack 64

.data

```

```
str1 db "Enter the first 2-digit number: $"
str2 db "Enter the second 2-digit number: $"
str3 db "Input1 is lesser than input2$"
str4 db "Input1 is greater than input2$"
str5 db "Input1 and Input2 are equal$"
newline db 13,10,"$"
input1 dw ?
input2 dw ?
mult10 db 10
```

```
.code
main:
mov ax, @data
mov ds, ax
```

```
mov ah, 09
lea dx, str1
int 21h
mov ah, 01
int 21h
```

```
sub al, "0"
```

```
mov ah, 0
mul mult10
mov input1, ax
```

```
mov ah, 01
int 21h
```

```
sub al, "0"
```

```
mov ah, 0
add input1, ax
```

```
mov ah, 09
lea dx, newline
int 21h
```

```
mov ah, 09
lea dx, str2
int 21h
mov ah, 01
int 21h
```

```
sub al, "0"
```

```
mov ah, 0
mul mult10
mov input2, ax
```

```
mov ah, 01
int 21h
```

```
sub al, "0"
```

```
mov ah, 0
add input2, ax
```

```
mov ah, 09
lea dx, newline
int 21h
```

```
mov ax, input1
mov bx, input2
cmp ax, bx
jne notEqual
```

```
mov ah, 09
lea dx, str5
int 21h
```

```
jmp endProgram
```

```
notEqual:
    cmp ax, bx
    ja greater
```

```
    mov ah, 09
    lea dx, str3
    int 21h
```

```
    jmp endProgram
```

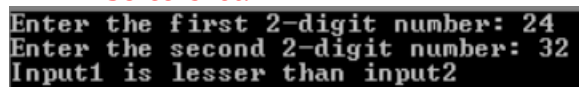
```
greater:
    mov ah, 09
    lea dx, str4
    int 21h
```

```
endProgram:
```

```
mov ah, 4ch
int 21h
```

```
end main
```

Screenshot:



```
Enter the first 2-digit number: 24
Enter the second 2-digit number: 32
Input1 is lesser than input2
```